

Ejercicios Guiados para programar en Rstudio: Metropolis-Hastings

Parte I: Entendiendo la Caminata Aleatoria

Ejercicio 1: Caminata Aleatoria Unidimensional

Considere una caminata aleatoria simple en $\mathbb{R}$:

$$\theta^{(m+1)} = \theta^{(m)} + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2) \quad (1)$$

- (a) Simule 1000 pasos de esta caminata con $\theta^{(0)} = 0$ y $\sigma = 0.1$.
- (b) Grafique la trayectoria. ¿Qué observa?
- (c) Repita con $\sigma = 1.0$. Compare ambas trayectorias.
- (d) **Reflexión:** ¿Cómo afecta σ a la exploración del espacio?

Ejercicio 2: Caminata Aleatoria multidimensional

Repetir el ejercicio anterior en dimensiones mayor que 1.

Ejercicio 2: Modelo Poisson con Metropolis-Hastings

Usando el modelo Poisson con prior no-informativo, implementar el ejercicio propuesto en este enlace:

[https://www.y1zhou.com/series/bayesian-stat/bayesian-stat-bayesian-inference-
?utm_source](https://www.y1zhou.com/series/bayesian-stat/bayesian-stat-bayesian-inference/?utm_source)

Ejercicio 2: De Caminata Aleatoria a Metropolis-Hastings

Contexto: Tenemos datos $y = \{6.2, 4.8, 5.1, 4.9, 5.3\}$ de una distribución $\mathcal{N}(\mu, 1)$ con varianza conocida. Queremos estimar μ .

Configuración Bayesiana:

$$\text{Verosimilitud: } y_i \sim \mathcal{N}(\mu, 1) \quad (2)$$

$$\text{Prior: } \mu \sim \mathcal{N}(0, 10) \quad (\text{prior débil}) \quad (3)$$

$$\text{Posterior: } \mu|y \sim \mathcal{N}(\mu_{post}, \sigma_{post}^2) \quad (4)$$

donde (por conjugación):

$$\mu_{post} = \frac{10 \cdot \bar{y} + 1 \cdot 0}{10 + 1} = \frac{10\bar{y}}{11} \quad (5)$$

$$\sigma_{post}^2 = \frac{1 \cdot 10}{10 + 5} = \frac{10}{15} = \frac{2}{3} \quad (6)$$

(a) **Paso 1:** Calcule analíticamente μ_{post} y σ_{post}^2 con los datos dados.

(b) **Paso 2:** Implemente la función de log-densidad posterior:

$$\ell(\mu) = \sum_{i=1}^n \left[-\frac{1}{2}(y_i - \mu)^2 \right] - \frac{1}{2 \cdot 10} \mu^2 + \text{const} \quad (7)$$

(c) **Paso 3:** Compare tres enfoques para generar muestras:

- **Caminata pura:** Acepta todos los pasos $\mu^{(m+1)} = \mu^{(m)} + \epsilon$
- **Metropolis-Hastings:** Usa mecanismo de aceptación/rechazo
- **Muestreo directo:** Muestrea directamente de $\mathcal{N}(\mu_{post}, \sigma_{post}^2)$

(d) **Paso 4:** Para MH, compare tasas de aceptación con $\sigma_{prop} \in \{0.1, 0.5, 1.0, 2.0\}$.

(e) **Pregunta clave:** ¿Por qué MH converge a la posterior correcta mientras que la caminata pura no?

Parte II: Casos Multidimensionales

Ejercicio 3: Normal Bivariada

Objetivo: Muestrear de $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ con:

$$\boldsymbol{\mu} = \begin{pmatrix} 2 \\ 3 \end{pmatrix}, \quad \boldsymbol{\Sigma} = \begin{pmatrix} 1 & 0.5 \\ 0.5 & 2 \end{pmatrix} \quad (8)$$

Ejercicio 3: Normal Bivariada

Objetivo: Muestrear de $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ con:

$$\boldsymbol{\mu} = \begin{pmatrix} 2 \\ 3 \end{pmatrix}, \quad \boldsymbol{\Sigma} = \begin{pmatrix} 1 & 0.5 \\ 0.5 & 2 \end{pmatrix} \quad (9)$$

- (a) Use matriz de propuesta $\boldsymbol{\Sigma}_{prop} = c \cdot \mathbf{I}_2$ con $c \in \{0.1, 0.5, 1.0, 2.0\}$.
- (b) Ejecute 10,000 iteraciones para cada valor de c .
- (c) Grafique las trayectorias y compare tasas de aceptación.
- (d) Calcule las medias y covarianzas muestrales. Compare con los valores teóricos.

Parte III: Transformaciones de Parámetros

Ejercicio 4: Parámetro con Restricción Positiva

Queremos muestrear de una distribución Gamma:

$$\theta \sim \text{Gamma}(\alpha = 2, \beta = 1), \quad \theta > 0 \quad (10)$$

- (a) **Enfoque directo:** Implemente MH directamente en $\theta > 0$, rechazando propuestas negativas.
- (b) **Con transformación:** Use $\xi = \log(\theta)$, implemente MH en $\xi \in \mathbb{R}$.
- (c) Compare eficiencia: tasas de aceptación, mezcla, tiempo de autocorrelación.

Parte IV: Aplicación Epidemiológica

Ejercicio 5: Modelo SIR Simplificado

Proyecto Final: Implementación Propia

Desarrolle su propia clase `MetropolisHastings` que incluya:

- Soporte para transformaciones automáticas de parámetros
- Calibración adaptativa de la matriz de propuesta durante burn-in

- Cálculo automático de diagnósticos de convergencia
- Visualización de resultados (trace plots, posteriores, etc.)
- Manejo robusto de errores numéricos

Pruebe su implementación en al menos tres problemas diferentes y compare con implementaciones estándar (ej. PyMC, Stan).